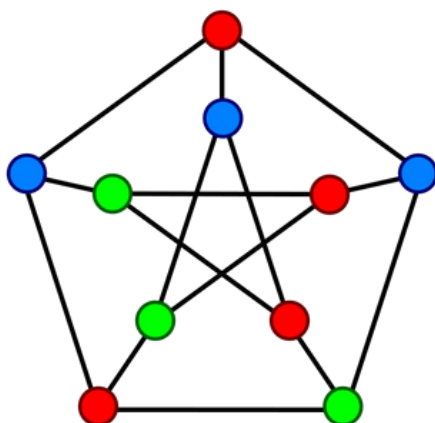

O problema do número cromático



Elementos de Programação *Projeto de Computação Evolutiva*

Docente: Carlos Caleiro



Grupo 6:

109316 – Joana Costa

109298 – Leonor Castelo

109604 – Marta Silva

109855 – Rúben Reis

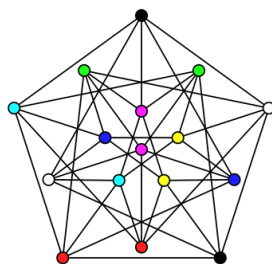
Outubro 2023

Índice

• Introdução	3
• Planificação	4
Grafos	4
Coloração	5
Indivíduos	6
População	6
Eventos	8
CAP	8
Randow e Math	8
Exprandow	8
• Simulador	9
• Tipos de eventos.....	10
Avaliação	10
Evolução	10
Seleção	11
• Resultados e Discussão.....	12
• Conclusão.....	13

1 - Introdução

Uma coloração de um grafo (não direcionado) diz-se ótima quando colore os vértices de modo que vértices conectados por uma aresta tenham cores diferentes. O seu número cromático corresponderá ao menor número de cores necessário para que o supramencionado se verifique.



Atualmente, a coloração de grafos modela alguns problemas com uma aplicação prática na sociedade. Destacam-se, por exemplo, a programação de horários e de períodos de avaliações tanto no ensino regular como no universitário, a alocação de recursos, o design de circuitos e a coloração de mapas geopolíticos.



Particularmente, a descoberta do número cromático de um grafo revela-se um problema de otimização combinatória NP-Difícil, isto é, pode ser um desafio complexo e demorado em termos computacionais e, atualmente, não existe qualquer algoritmo que o resolva numa grande escala e em tempo útil. Por esta razão, existem alguns métodos aproximados que formam um conjunto de técnicas que permitem economizar algum tempo na procura de uma boa estimativa do número cromático, poupando recursos computacionais.

Desenvolver um programa em Python para resolver o problema do número cromático usando técnicas de computação evolutiva permite não apenas obter uma solução aproximada, mas também explorar e aplicar algoritmos de inspiração na evolução biológica no processo de otimização. Assim, a implementação em Python permite a rápida prototipagem e experimentação com diferentes técnicas de resolução, de modo a compreender e aprimorar os métodos para resolver os problemas relacionados com a coloração dos grafos, sobretudo quando estes são complexos e se torna bastante difícil determinar o número cromático.

2 - Planificação

Para permitir implementar os diversos tipos de dados num simulador, é necessário ter em conta as operações que são necessárias dentro dos tipos de dados.

Grafos

- **newgraph (N)** — Uma função que dado um número, N , permite criar um grafo com esse número de nós;
- **edge_index (g, x, y)** — Uma função que, dado um grafo g e uma aresta que une o nó x ao y , vai analisar se essa aresta pertence ao grafo;
- **add_edge (g, x, y)** — Uma função que dado um grafo, g , e uma aresta que une o nó x ao y , permite adicionar, se ainda não existir, essa aresta;
- **deledge (g, x, y)** — Uma função que dado um grafo g e uma aresta, que une o nó x ao nó y , devolve o grafo sem essa aresta;
- **dim (g)** — Uma função que, dado um grafo, g , devolve o seu número de nós;
- **graphQ (g)** — Uma função que, ao receber um argumento, avalia se este corresponde um grafo ou não;
- **list_nodes (g)** — Uma função que recebe um grafo g , percorre as suas arestas e devolve a lista / dos seus nós;
- **emptyQ (g)** — Uma função que recebe um grafo e avalia se este está vazio ou não, ou seja, se existem arestas a ligar os nós ou não;
- **edge (g, x, y)** — Uma função que dado um grafo, g , e a aresta que liga o nó x ao y , avalia se a aresta pertence ao gráfico.

```
def newgraph(N):
    return [N, []]

def edge_index(g, x, y):
    i = 0
    notFound = True
    while notFound and i < len(g[1]):
        notFound = (g[1][i] != (x, y) or g[1][i] != (y, x))
        i = i + 1
    if not notFound:
        i = i - 1
    return i

def addedge(g, x, y):
    i = edge_index(g, x, y)
    if i == len(g[1]):
        return [g[0], g[1] + [(x, y)] + [(y, x)]]
    else:
        return g

def deledge(g, x, y):
    i = edge_index(g, x, y)
    return [g[0], g[1][:i-1] + [1][i:]]

def dim(g):
    return g[0]

def graphQ(g):
    if type(g) == list:
        if len(g) == 2:
            if type(g[0]) == int:
                if type(g[1]) == list:
                    i = 0
                    notFound = True
                    while notFound and i < len(g[1]):
                        notFound = len(g[1][i]) != 2
                        i = i + 1
                    if notFound:
                        return len(list_nodes(g)) <= g[0]
                    else:
                        return False
                else:
                    return False
            else:
                return False
        else:
            return False
    else:
        return False

def list_nodes(g):
    l = []
    for e in g[1]:
        if not g[0] in l:
            l = l + [g[0]]
        if not g[1] in l:
            l = l + [g[1]]
    return l

def emptyQ(g):
    return len(g[1]) == 0

def edge(g, x, y):
    return (x, y) in g[1] or (y, x) in g[1]
```

Coloration

- **col (G)** — Uma função que dado um grafo, G , devolve uma lista com números (em que cada um simboliza uma cor diferente), em que cada índice representa um nó;
- **viable_col (G, ind)** — Uma função que dado um grafo G e uma coloração col , vê se essa coloração é válida, ou seja, não há dois ou mais nós da mesma cor ligados por uma aresta;
- **procura (n, w)** — uma função auxiliar que recebe uma lista w e um número n e verifica se esse número está na lista

```
def procura(n,w):
    found=False
    i=0
    while i<len(w) and not found:
        if w[i]==n:
            found=True
            i=i+1
    return found
```

#função que vai contar o número de cores do grafo G

```
def num_cores(ind):
    mylist=[]
    res=len(mylist)
    h=individuals.col_ind(ind)
    i=0
    while i<len(h):
        if not procura(h[i],mylist):
            mylist=mylist+[h[i]]
            res=res+1
            i=i+1
        mylist=mylist
    return res
```

#função que vai ver o número de defeitos de coloração

```
def def_col_count(G,ind):
    m=individuals.col_ind(ind)
    if viable_col(G,ind):
        return 0
    else:
        res=0
        i=0
        for i in range(len(m)-1): #vai averiguar se a cor de
            if m[i]==m[i+1]:
                res=res+1
        return res
```

```
import individuals
import grafos
```

#coloração

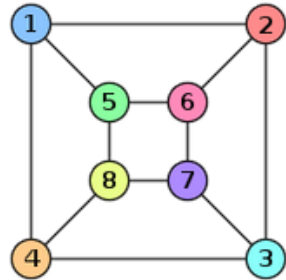
#coloração

```
def col(G):
    N=grafos.dim(G)
    res=[]
    i=1
    while i<N+1:
        res=res+[i]
        i=i+1
    return res
```

#Uma função que vai ver se a coloração(col) é possível

```
def viable_col(G,ind):
    found=False
    col=individuals.col_ind(ind)
    x=0
    while x<len(col) and not(found):
        y=1
        while y<len(col) and not(found):
            if grafos.edge(G,x,y) and col[x]==col[y]:
                found=True
            else:
                y=y+1
        x=x+1
    return not(found)
```

- **num_cores (ind)** — Uma função que permite, dado um indivíduo ind , ver com quantas cores diferentes está pintado;
- **def_col_count (G, col)** — Uma função que recebe como variáveis um grafo G e uma coloração col e calcula, se a coloração for inválida, o número de defeitos do grafo, ou seja, o número de arestas que ligam nós da mesma cor.



Individuais

- **ind (G, pop, ct)** — Uma função que dado um grafo *G*, uma população *pop* e um tempo *ct*, devolve uma lista cujo primeiro elemento é a sua coloração, o segundo é a data de nascimento e, por último, nessa lista surge o número de identificação, que é único;
- **n_ind (ind)** — Uma função que, dado um grafo, devolve o seu número identificador;
- **idade_ind (ind, ct)** — Uma função que recebe um indivíduo e um instante e devolve a sua idade nesse momento;
- **col_ind (ind)** — Uma função que recebe um indivíduo e devolve a sua coloração;
- **coe_adap (G, ind)** — uma função que recebe o grafo original e um indivíduo e devolve o coeficiente de adaptação do indivíduo.
- **criar_ind (G, ind, pop, ct)** — Uma função que recebe um grafo, uma população, um indivíduo e um instante de tempo e retorna um indivíduo novo, concebido através da mutação do indivíduo que a função recebeu.
- **mutacao_ind(G, ind)** — Uma função que recebe um grafo *G* e um indivíduo e devolve um indivíduo mutado

```
import pop
import coloration
import random

def novo_ind(G,pop,tempo):
    res=[coloration.col(G),tempo,pop.ultimo_num(pop)+1]
    pop=pop.add_ind(pop,res)
    return res

def col_ind(ind):
    return ind[0]

def n_ind(ind):          #ler o número que identifica cada indivi
    return ind[2]

def idade_ind(ind,ct):   #idade do individuo
    return ct-ind[1]

def col_ind(ind):        #coloração do individuo
    return ind[0]

def coe_adapt(G,ind):    #coeficiente de adaptação
    if not(coloration.viable_col(G,ind)):
        return 1/(1+coloration.def_col_count(G,ind))
    else:
        return coloration.dim(G)/coloration.num_cores(ind)

def criar_ind(G,ind,pop,ct):
    if pop==[]:
        ultimo_num(pop)==0
        i=random.randrange(len(ind[0]))
        k=random.choice(ind[0])
        ind1=[[],0,0]
        ind1[0]=ind[0][:i]+[k]+ind[0][i+1:]
        ind1[1]=ct
        ind1[2]=ultimo_num(pop)+1
        pop=pop.add_ind(pop,ind1)
        return ind1

def mutacao_ind(G,ind,pop):
    l=col_ind(ind)
    i=random.randrange(len(l))
    k=random.choice(l)
    ind1=[[],0,0]
    ind1[0]=ind[0][:i]+[k]+ind[0][i+1:]
    ind1[1]=ind[1]
    ind1[2]=ind[2]
    if coloration.coe_adapt(G,ind1)>=coloration.coe_adapt(G,ind):
        return ind1
    else:
        return ind
```

Pop (População)

- **empty_pop ()** — Uma função que avalia se a população está vazia, ou seja, sem indivíduos;
- **pop_inicial (G, K, empty_pop)** — Uma função que, ao receber um grafo, *G*, um número, *K*, e uma população vazia, devolve uma população com *K* indivíduos, todos com a mesma coloração uniforme (ou seja, uma cor distinta para cada nó de *G*). Essa população será representada por uma lista, na qual o primeiro elemento é um conjunto de sublistas (cada uma representa um indivíduo) e o segundo elemento é uma lista com os números identificadores de todos os indivíduos, por ordem crescente de data de nascimento;

```
import individuals
import coloration
import grafos

#funções ligadas à População

def empty_pop():
    return []

def pop_inicial(pop):
    pop=empty_pop()
    i=0
    for i in range(k):
        new_ind=individuals.novo_ind(G,pop,0)
    return pop

def lista_ind(pop):
    return pop

def num_ind(pop):
    return len(pop)

def ultimo_num(pop1):
    if len(pop1)==0:
        return 0
    else:
        return pop1[-1][-1]
```

- **num_ind (pop)** — Uma função que recebe uma população e devolve o número de indivíduos que lá estão inseridos;
- **add_ind (pop, ind)** — Uma função que recebe uma população e um indivíduo, adicionando-o à população em questão e devolvendo a população atualizada;
- **del_ind1 (pop, ind)** — Uma função que recebe uma população e um indivíduo, eliminando-o da população em que está inserido, devolvendo a população atualizada;
- **existe_ind (pop, ind)** — Uma função que, ao receber uma população e um indivíduo, averigua se este indivíduo está inserido na população;
- **melhor_ind (pop)** — Uma função que recebe uma população e devolve o indivíduo com o melhor coeficiente de adaptação;
- **morte (G, pop)** — Uma função que recebe uma população e um grafo e devolve a população após eliminar todos os indivíduos que tenham uma coloração não viável;
- **morte1 (G, pop)** — Uma função que recebe um grafo e uma população e elimina o indivíduo com menor coeficiente de adaptação (se houver mais do que um com o menor, elimina o indivíduo mais velho);
- **lista_coe (G, pop)** — Uma função que recebe um grafo e uma população e devolve a lista dos coeficientes de adaptação de cada um dos indivíduos;
- **bubble_sort (w)** — Uma função que recebe uma lista e devolve essa lista com os números ordenados por ordem crescente.

```
def add_ind(pop, ind):
    if pop is None:
        pop=[]
    pop.append(ind)
    return pop

def del_ind(pop,ind):
    new_pop=[]
    for individual in pop:
        if individual!=ind:
            new_pop.append(individual)
    return new_pop

def existe_ind(pop,ind):
    found=False
    i=0
    while i<len(pop) and not found:
        ind1=pop[i]
        if individuals.n_ind(ind1)==individuals.n_ind(ind):
            found=True
        i=i+1
    return found

def melhor_ind(pop):
    if len(pop) == 0:
        return None
    best_ind=pop[0]
    best_adapt=individuals.coe_adapt(G,best_ind)
    for ind in pop[1:]:
        cur_coe_adapt=individuals.coe_adapt(G, ind)
        if cur_coe_adapt>best_adapt:
            best_ind=ind
            best_adapt=cur_coe_adapt
    return best_ind

def morte(G, pop):
    res=empty_pop()
    for i in range(len(pop)):
        if individuals.viable_col(G,pop[i]):
            res=add_ind(res,pop[i])
    return res

def morte1(G, pop):
    pop1=pop[:]
    coe_values=lista_coe(G, pop1)
    min_coe=bubble_sort(coe_values)[0]
    found=False
    i=0
    while i < len(pop1) and not found:
        if individuals.coe_adapt(G, pop1[i])==min_coe:
            pop1=del_ind(pop1, pop1[i])
            found=True
        else:
            i=i+1
    return pop1

def lista_coe(G,pop):
    list1=[]
    i=0
    for i in range(len(pop)):
        list1=list1+[individuals.coe_adapt(G,pop[i])]
    return list1

def bubble_sort(w):
    n=len(w)
    for i in range(n):
        for j in range(0,n-i-1):
            if w[j]>w[j+1]:
                w[j],w[j+1]=w[j+1],w[j]
    return w
```

Event

- **evt (t, k, individual)** — Uma função que recebe um instante, t, um tipo de evento, k, e um indivíduo e devolve uma lista com cada um desses elementos;
- **time (e)** — Uma função que, ao receber um evento, e, devolve o instante em que o evento se irá realizar;
- **kind (e)** — Uma função recebe um evento, e, retorna a categoria do evento.
- **individual (e)** — Uma função que recebe um evento e devolve o indivíduo referente ao evento

```
def evt(t,k,ind):  
    return [t,k,ind]  
  
def time(e):  
    return e[0]  
  
def kind(e):  
    return e[1]  
  
def individual(e):  
    return e[2]
```

CAP - Cadeia de acontecimentos pendentes

- **newc ()** — Uma função que devolve uma nova cadeia de acontecimentos pendentes, representada por uma lista vazia;
- **addE (c, e)** — Uma função que insere um evento (e) numa CAP (c) na posição correspondente, tendo em conta o seu atributo *time*;
- **delE (c, e)** — Uma função que recebe uma CAP, c, e apaga o primeiro evento, ou seja, o primeiro elemento da lista;
- **nextE (c)** — Uma função que recebe uma CAP, c, e retorna o primeiro elemento da lista.
- **showE (c)** — Uma função que recebe uma CAP, c, e devolve a categoria, o indivíduo em questão e o instante em que cada evento se irá realizar.

```
import event  
  
def newc():  
    return []  
  
def addE(c,e):  
    return [el for el in c if event.time(el)<event.time(e)]+[e]+  
           [el for el in c if event.time(el)>event.time(e)]  
  
def delE(c,e):  
    if len(c)>0:  
        return c[1:]  
    else:  
        print("Erro! A cap está vazia")  
  
def nextE(c):  
    if len(c)>0:  
        return c[0]  
    else:  
        print("Erro! A cap está vazia")  
  
def showE(c):  
    for e in c:  
        print(event.time(e), event.kind(e),event.individual(e))
```

Random e math

No decorrer da simulação tivemos de recorrer ao módulo *random* e *math* para conseguirmos aceder à função *random* e a expressões matemáticas, tais como o *atan* e o número de nepler.

Exprandom

Criámos também um módulo (*exprandom.py*) que ao receber o módulo *random* e o módulo *math*, retorna uma observação da variável aleatória exponencial de valor médio m.

```
import random  
import math  
  
def exprandom(m):  
    x=random.random()  
    return -m*math.log(x)
```

3-Simulador

Antes de mais, para ser possível utilizar os módulos que criámos foi necessário importá-los. Decidimos que, para avaliar a evolução da população, iríamos querer receber no final um gráfico que ilustrasse o número de indivíduos na população, em função do tempo.

```
import coloration
import cap
import pop
import grafos
import event
import individuals
import exprandom
import math
import random
import matplotlib.pyplot as plt
```

O simulador vai receber 6 parâmetros:

- O tempo limite até ao qual a simulação pode decorrer (ht);
- O tempo que irá definir a cadência do evento de evolução (Tri);
- O tempo que ditará a cadência do evento de avaliação (Tlim);
- O tempo que dirá a cadência do evento de seleção (Tfil)
- O grafo em questão (G);
- O número de indivíduos que terá a população inicial (k).

O simulador também irá contar com as variantes:

- ce: current event (referente ao evento que estará em simulação);
- ct: current time (instante em que o evento está a decorrer), que será zero no início da simulação;
- ck: current kind (o tipo de evento que está a ser simulado);
- ci: current indivíduo (o indivíduo ao qual está a ser aplicado o evento atual).

```
def sim(ht,tri,tlim,tfil,G,k):
    c=cap.newc()
    population=pop.pop_initial(G,k)
    ct=0
    c=cap.addE(c,event.evt(ct+tfil,"sel",population))

    for ind in pop.lista_ind(population):
        c=cap.addE(c,event.evt(ct+exprandom.exprandom(tri),"ev",ind))
        c=cap.addE(c,event.evt(ct+exprandom.exprandom(tlim),"av",ind))

    xtrace = [0]
    ytrace = [k]

    while ct<=ht:
        ce=cap.nextE(c)
        ct=event.time(ce)
        ck=event.kind(ce)
        ci=event.individual(ce)
```

O simulador iniciará um ciclo com o intuito de encontrar a melhor aproximação para a solução do problema do número cromático. Para começar, criámos uma agenda e calendarizámos os 3 tipos de eventos para cada um dos indivíduos presentes na população inicial como o objetivo de eliminar, reproduzir ou mutar cada um deles, dependendo do evento atual, de modo que consigamos o pretendido.

Cada tipo de evento foi marcado tendo em conta as variáveis aleatórias exponenciais de valor médio associadas a cada um deles: na avaliação o valor médio será *Tlim*, na evolução será *Tri* e na seleção será *Tfil*.

4 - Tipos de Eventos

Avaliação

Resulta na morte do indivíduo, se ainda for vivo nesse instante, se o número dado pela função `random()`, que devolve um valor real escolhido uniformemente no intervalo $[0, 1[$ for menor do que o valor da probabilidade dado pela fórmula:

$$1 - \frac{2}{\pi} \arctan((1 + A)^{1 + \frac{8}{1+I}})$$

onde A é o coeficiente de adaptação do indivíduo nesse instante, e I é a sua idade;

```
if ck=="av" and pop.existe_ind(population,ci):
    A=individuals.coe_adapt(G,ci)
    I=individuals.idade_ind(ci,ct)
    q=1-(2/math.pi)*math.atan((1+A)**(1+(8/(1+I))))
    l = random.random()
    if l<q:
        popl=pop.del_ind(population,ci)
    else:
        c=cap.addE(c,event.evt(ct+exrandom.exrandom(tlim),"av",ci))
```

Evolução

O evento de evolução é um processo que impacta um indivíduo específico e ocorre com uma frequência determinada por uma variável aleatória exponencial, com um valor médio denominado *Tri*. Se o indivíduo ainda estiver vivo quando o evento ocorre, duas possibilidades surgem: uma mutação com uma probabilidade *Prob* ou reproduz-se com uma probabilidade igual a $1 - Prob$.

A probabilidade *Prob* é calculada do seguinte modo, tendo em conta que K é o número inicial de indivíduos na população e T é o tamanho da população no instante que ocorre no evento:

$$\frac{1}{1 + e^{\frac{K-T}{10}}}$$

Na ocorrência de uma mutação, a coloração do indivíduo é alterada em apenas um nó, selecionado uniformemente, ou seja, essa cor é escolhida igualmente de entre as cores já presentes na sua coloração atual. É notório salientar que essa mudança terá impacto somente se a nova coloração resultar numa melhoria do coeficiente de adaptação do indivíduo.

```
elif ck=="ev" and pop.existe_ind(population,ci):
    l=pop.num_ind(population)
    m=1/(1+(math.e)**((k-l)/10))
    n=random.random()
    if n<m:
        ind2=individuals.mutacao_ind(G,ci,population)
        if ind2==ci:
            c=cap.addE(c,event.evt(ct+exrandom.exrandom(tri),"ev",ci))
        else:
            population=pop.del_ind(population,ci)
            population=pop.add_ind(population,ind2)
            c=cap.addE(c,event.evt(ct+exrandom.exrandom(tri),"ev",ind2))
            c=cap.addE(c,event.evt(ct+exrandom.exrandom(tlim),"av",ind2))
    else:
        ind3=individuals.criar_ind(G,ci,population,ct)
        c=cap.addE(c,event.evt(ct+exrandom.exrandom(tri),"ev",ci))
        c=cap.addE(c,event.evt(ct+exrandom.exrandom(tri),"ev",ind3))
        c=cap.addE(c,event.evt(ct+exrandom.exrandom(tlim),"av",ind3))
```

Seleção

Este é um evento que influencia uma população, com uma cadência representada pela constante *T_{fil}*. Quando ocorre, leva à morte de todos os indivíduos cuja coloração, naquele instante, é considerada inválida. Se a coloração de todos os indivíduos da população já for válida, este evento pode também ocasionar a morte dos indivíduos que possuem os piores coeficientes de adaptação. Morrerão, neste caso, tantos quantos necessários para que a população resultante não exceda 3/2 do tamanho da população inicial (*K*).

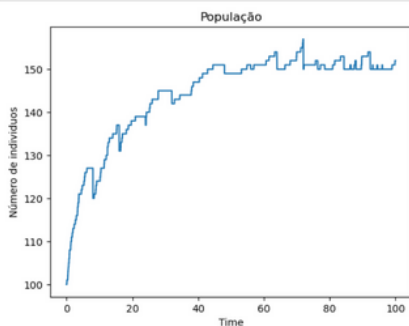
```
else:
    population=pop.morte(G,population)
    while pop.num_ind(population)>3*K//2:
        population=pop.mortel(G,population)
    c=cap.add(c,event.evt(ct+tfil,"sel",population))

xtrace.append(ct)
ytrace.append(pop.num_ind(population))
c=cap.del(c,ce)
ce=cap.next(c)
ct=event.time(ce)
ck=event.kind(ce)
ci=event.individual(ce)

plt.plot(xtrace, ytrace)
plt.xlabel('Time')
plt.ylabel('Número de indivíduos')
plt.title('População')
plt.show()
print(pop.melhor_ind(G,population))
return coloration.num_cores(pop.melhor_ind(G,population))
```

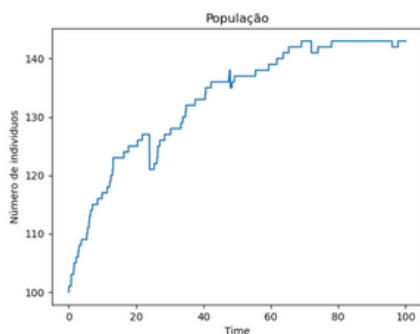
Para a simulação continuar é necessário eliminar o evento que já ocorreu e ler o próximo evento na agenda e executá-lo

5 - Resultados e discussão



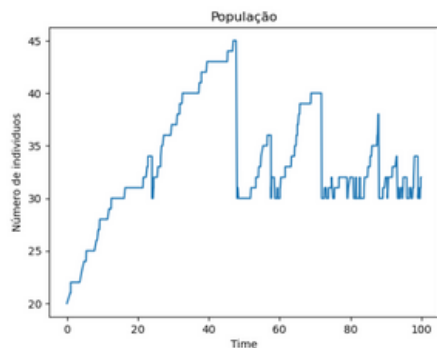
[[6, 2, 3, 6, 3, 6, 3, 3, 2, 2], 54.848217947431124, 101]

3



[[1, 9, 3, 7, 9, 3, 7, 7, 9, 9], 6.228269001491079, 31]

4



[[1, 2, 1, 4, 2, 6, 7, 7, 2, 2], 0, 14]

5

1

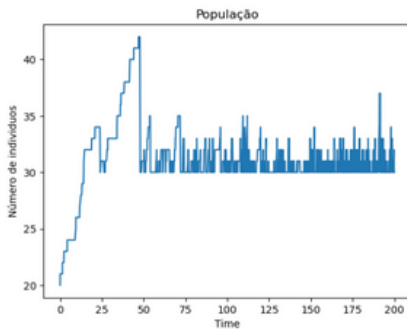
Para testar o simulador decidimos utilizar o grafo de Petersen, cujo número cromático é conhecido (3), para conseguirmos saber qual o valor desejado no final da simulação

2

Neste exemplo, ao manter o tempo limite e o número de indivíduos inicial e aumentando para o triplo o *Tri*, o *Tlim*, o *Tfi*, o resultado já não é o pretendido, ou seja, ao aumentar a cadência dos eventos, o simulador demora mais algum tempo a conseguir alcançar o resultado pretendido.

3

Nesta simulação, mantivemos os valores do exemplo 2 e diminuímos o número de indivíduos da população inicial, o que, tal como planeado, irá fazer com que seja necessário mais tempo para atingir o resultado desejado



[[10, 7, 10, 9, 7, 10, 7, 9, 9, 10], 150.81973993540726, 4]

3

4

Nesta simulação, a única variável que alterámos, relativamente ao exemplo 3, foi o tempo limite, ou seja, o tempo máximo de simulação, cujos resultados vão de encontro ao que nós esperávamos.

6- Conclusão

Durante o desenrolar deste projeto, explorámos e consolidámos profundamente não só os conhecimentos fundamentais da linguagem de programação Python, mas também desenvolvemos técnicas para obter soluções aproximadas com um impacto significativo na resolução de problemas de elevado grau de complexidade. Estes compreendem uma gama diversificada de problemas práticos, tanto de natureza matemática como de outras áreas, cuja resolução é essencial para o nosso quotidiano.

Particularmente, o problema do número cromático, cuja tentativa de solução aproximada foi descrita neste relatório, é um exemplo que reflete a falta de um algoritmo eficaz para alcançar a sua solução exata, dado a sua extrema dificuldade. Logo, entende-se porque este se trata de um problema NP-Difícil.

Em resposta a essa dificuldade, adotámos o paradigma da computação evolutiva e empregámos técnicas de simulação estocástica na resolução deste problema. Esta abordagem permitiu a obtenção de soluções que, apesar de aproximadas, são bastante úteis, revelando a grande eficácia dos métodos heurísticos para enfrentar problemas intratáveis com a maior eficiência e precisão possível. Isto deve-se, sobretudo, à flexibilidade e adaptabilidade que estes métodos têm para lidar com a complexidade deste tipo de problemas.

Por último, a resolução aproximada do número cromático usando técnicas evolutivas e estocásticas permitiu que houvesse uma aplicação bem-sucedida de conceitos complexos, que, por sua vez, possibilitou o enriquecimento da nossa capacidade de compreensão de algoritmos e estruturas de dados.